

ASSIGNMENT 15

ROLL NO:CSB24083

NAME:DIPANKAR BEZBARUAH

Without any synchronization

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

int counter = 0;

void* increment_counter(void* arg)
{
    for(int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    // Create threads
    for(int i = 0; i < NUM_THREADS; i++)
    {
```

```

        pthread_create(&threads[i], NULL, increment_counter,
NULL);
    }

    // Wait for threads to finish
    for(int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final Counter Value: %d\n", counter);

    return 0;
}

```

OUTPUT

```

PS F:\AP projects> gcc -g -pthread multithread1_2.c -o multithread1_2
PS F:\AP projects> .\multithread1_2.exe
Final Counter Value: 143307

```

Using mutex

```

#include<stdio.h>
#include<pthread.h>
#define NUM_THREADS 4
#define INCREMENTS 100000
int counter=0;
pthread_mutex_t lock;
void*increment_counter(void*arg)
{
    for(int i=0;i<INCREMENTS;i++)

```

```

    {
        pthread_mutex_lock(&lock);
        counter++;          // Critical section
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
int main(){
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&lock,NULL);
    for(int i=0;i<NUM_THREADS;i++){
        pthread_create(&threads[i],NULL,increment_counter,NULL);
    }
    for(int i=0;i<NUM_THREADS;i++){
        pthread_join(threads[i],NULL);
    }
    printf("final counter value: %d\n",counter);

    pthread_mutex_destroy(&lock);
    return 0;
}

```

OUTPUT

```

PS F:\AP projects> gcc -g -pthread multithread.c -o multithread
PS F:\AP projects> .\multithread.exe
final counter value: 400000

```

Why Race Condition Occurs

The statement:

```
counter++;
```

is not atomic. Internally it happens in three steps:

Read counter value from memory

Increment it

Write it back to memory

Suppose:

Thread A reads counter = 5

Thread B also reads counter = 5

Both increment to 6

Both write back 6

Then one increment is lost.

So multiple threads access and modify the shared variable simultaneously, causing incorrect results. This is called a race condition.

How Mutex Solves the Problem

A mutex provides mutual exclusion.

When one thread executes:

```
pthread_mutex_lock(&lock);
```

other threads must wait until:

```
pthread_mutex_unlock(&lock);
```

is executed.

So only one thread can enter the critical section at a time. This prevents simultaneous modification of the shared variable and removes the race condition.